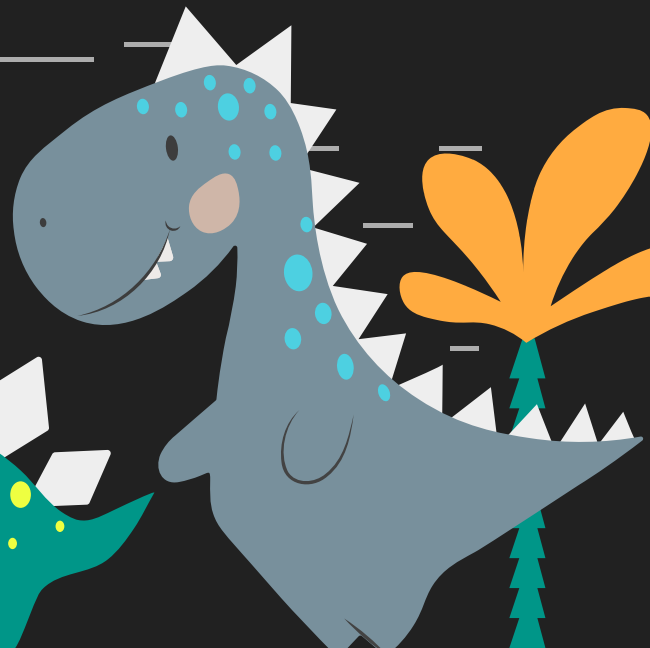
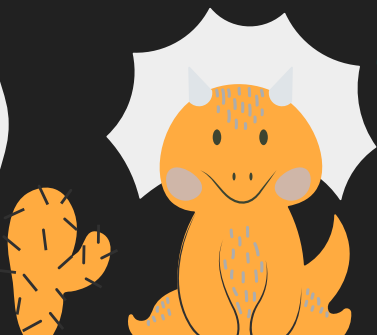
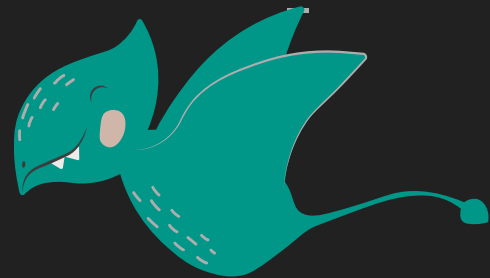


Summarizer

14.03.2023

Rui Godinho
#POWrangers





LinkedIn is a social network for professionals;

Companies use the Recruiter account enabling them to search the entire LinkedIn network;

Works better with keywords;

Great tool for devs;





About Me;

Work Experience;

Education;

Certifications;

Skill && Endorsements;

Accomplishments || Projects;

Languages;

Interests;

and everything you think it will be useful to get you a better
and more attractive profile!





Rui Godinho

<Code Cadet> @ <Academia de Código_>

Lisboa, Lisbon, Portugal · [Contact info](#)

233454543223442 connections

Open to

Add profile section

More



<Academia de Código_>

Create Visibility, update your profile and connect, connect, connect...





Introducing: the Accumulator of knowledge!!!!

Powerful tool to create builds and manage any Java-based project;

Manages dependencies → relationships between tasks;

Convention over configuration;

Easier to use;

Project Object Model → pom.xml

Contains information about project and configuration details

Useful commands:
mvn -version
mvn package
mvn clean



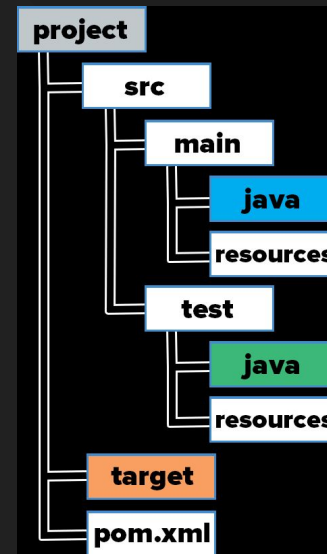


The most powerful features of maven:

create a standard project structure;

download the project dependency libraries automatically;

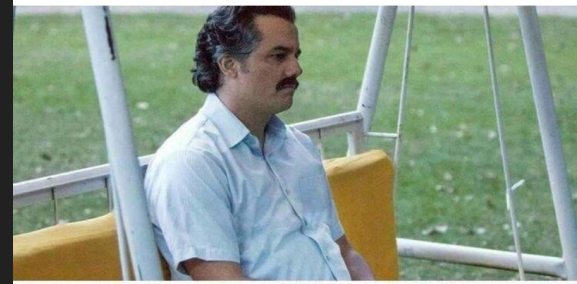
save them into local repository (m2 folder);



DEFAULT PROJECT OBJECT MODEL (POM) BUILD LIFECYCLE

VALIDATE	COMPILE	TEST	PACKAGE	VERIFY	INSTALL	DEPLOY
Validate all necessary information for the project	Compile the source code	Test the compiled source code	Package up compiled source code and other resources	Verify package against criteria	Install the package into the local repository	Deploy the package to a remote repository

Me waiting for Maven to download every dependency after deleting the m2 folder



JavaBank exercise



Convert JavaBank into a Maven project;

Implement Service Layer Pattern:

Create 3 Interfaces:

CustomerService;

AccountService;

AuthService;

and eliminate the Bank Class and AccountManager Class, dividing them into:

3 new Classes:

ServiceCustomer;

ServiceAccount;

ServiceAuth;





```
public interface CustomerService {  
  
    Customer get(Integer id);  
  
    List<Customer> list();  
  
    Set<Integer> listCustomerAccountIds(Integer id);  
  
    double getBalance(int customerId);  
  
    void add(Customer customer);  
}
```



```
public interface AuthService {  
  
    boolean authenticate(Integer id);  
    Customer getAccessingCustomer();  
}
```



```
public interface AccountService {  
  
    void add(Account account);  
    void deposit(int id, double amount);  
    void withdraw(int id, double amount);  
    void transfer(int srcId, int dstId, double amount);  
}
```




```
public class LoginController extends AbstractController {

    private Controller nextController;
    private Bank bank;
    private ServiceAuth serviceAuth;

    public void setNextController(Controller nextController) {
        this.nextController = nextController;
    }

    public void setBank(Bank bank) {
        this.bank = bank;
    }

    public void onLogin(int id) {

        if(serviceAuth.authenticate(id)){
            //servic.setLoginCustomer(id);
            nextController.init();
        } else {
            view.show();
        }

    }

    public void setServiceAuth(ServiceAuth serviceAuth) {
        this.serviceAuth = serviceAuth;
    }

}
```



```
public class ServiceAccount implements AccountService{

    private AccountFactory accountFactory = new AccountFactory();
    private Map<Integer, Account> accountMap;
    Account account;

    public ServiceAccount(){
        this.accountMap = new HashMap<>();
    }

    @Override
    public void add(Account account) {
        accountMap.put(account.getId(), account);
    }

    @Override
    public void deposit(int id, double amount) {
        accountMap.get(id).credit(amount);
    }

    @Override
    public void withdraw(int id, double amount) {
        Account account = accountMap.get(id);

        if (!account.canWithdraw()) {
            return;
        }

        accountMap.get(id).debit(amount);
    }

    @Override
    public void transfer(int srcId, int dstId, double amount) {
    }

    public Account openAccount(AccountType accountType) {
        account = accountFactory.createAccount(accountType);
        accountMap.put(account.getId(), account);
        return account;
    }

}
```



```
public class ServiceAuth implements AuthService{

    ServiceCustomer serviceCustomer;
    Customer customer;

    @Override
    public boolean authenticate(Integer id) {
        if(serviceCustomer.get(id) != null){
            customer = serviceCustomer.get(id);
            return true;
        }
        return false;
    }

    @Override
    public Customer getAccessingCustomer() {
        return customer;
    }
    public void setServiceCustomer(ServiceCustomer serviceCustomer) {
        this.serviceCustomer = serviceCustomer;
    }
}
```



```
public class LoginView extends AbstractView {

    private LoginController loginController;

    public void setLoginController(LoginController loginController) {
        this.loginController = loginController;
    }

    @Override
    public void show() {
        showBankName();
        showLoginPrompt();
    }

    private void showBankName() {
        System.out.println("\n" + Messages.VIEW_LOGIN_WELCOME);
    }

    private void showLoginPrompt() {

        IntegerInputScanner scanner = new IntegerInputScanner();
        scanner.setMessage("\n" + Messages.VIEW_LOGIN_MESSAGE);
        scanner.setError(Messages.VIEW_LOGIN_ERROR);
        loginController.onLogin(prompt.getUserInput(scanner));
    }
}
```

```
public abstract class AbstractController implements Controller {

    protected View view;
    protected ServiceAuth serviceAuth;
    protected ServiceAccount serviceAccount;

    public void setView(View view) {
        this.view = view;
    }

    @Override
    public void init() {
        view.show();
    }

    public void setServiceAuth(ServiceAuth serviceAuth) {
        this.serviceAuth = serviceAuth;
    }

    public void setServiceAccount(ServiceAccount serviceAccount) {
        this.serviceAccount = serviceAccount;
    }
}
```

```
public class ServiceCustomer implements CustomerService {

    private HashMap<Integer, Customer> customers;
    int loginCustomer;

    public ServiceCustomer(){
        this.customers = new HashMap<>();
    }
    @Override
    public Customer get(Integer id) {
        return customers.get(id);
    }

    @Override
    public List<Customer> list() {
        return null;
    }

    @Override
    public Set<Integer> listCustomerAccountIds(Integer id) {
        return null;
    }

    @Override
    public double getBalance(int customerId) {
        return 0;
    }

    @Override
    public void add(Customer customer) {
        customers.put(customer.getId(), customer);
    }
}
```



```
public abstract class AbstractView implements View {

    protected Prompt prompt;
    protected Bank bank;
    protected ServiceAuth serviceAuth;

    public void setPrompt(Prompt prompt) {
        this.prompt = prompt;
    }

    public void setBank(Bank bank) {
        this.bank = bank;
    }

    public void setServiceAuth(ServiceAuth serviceAuth) {
        this.serviceAuth = serviceAuth;
    }
}
```



```
public class BalanceView implements View {

    private DecimalFormat df = new DecimalFormat("#.##");
    private Bank bank;
    private ServiceAuth serviceAuth;

    public void setBank(Bank bank) {
        this.bank = bank;
    }

    @Override
    public void show() {
        showBalance();
    }

    private void showBalance() {

        Customer customer = serviceAuth.getAccessingCustomer();
        System.out.println("\n" + customer.getName() + Messages.VIEW_BALANCE_MESSAGE + "\n");

        Set<Account> accounts = customer.getAccounts();
        for (Account account : accounts) {
            System.out.println(account.getId() + "\t" + account.getAccountType() + "\t" +
                df.format(account.getBalance()));
        }

        System.out.println("\n\n" + Messages.VIEW_BALANCE_TOTAL_MESSAGE +
            df.format(customer.getBalance()));
    }

    public void setServiceAuth(ServiceAuth serviceAuth) {
        this.serviceAuth = serviceAuth;
    }
}
```



```
public class DepositController extends AbstractAccountTransactionController {  
  
    @Override  
    public void submitTransaction(int accountId, double amount) {  
        serviceAccount.deposit(accountId, amount);  
    }  
}
```



```
public class WithdrawalController extends AbstractAccountTransactionController {  
  
    @Override  
    public void submitTransaction(int accountId, double amount) {  
        serviceAccount.withdraw(accountId, amount);  
    }  
}
```



```

public class MainTestezao {
    public static void main(String[] args) {

        ServiceAuth serviceAuth = new ServiceAuth();
        ServiceAccount serviceAccount = new ServiceAccount();
        ServiceCustomer serviceCustomer = new ServiceCustomer();
        serviceAuth.setServiceCustomer(serviceCustomer);

        Customer c1 = new Customer(1, "Rui");
        Customer c2 = new Customer(2, "Sergio");
        Customer c3 = new Customer(3, "Bruno");

        serviceCustomer.add(c1);
        serviceCustomer.add(c2);
        serviceCustomer.add(c3);

        Prompt prompt = new Prompt(System.in, System.out);

        LoginController loginController = new LoginController();
        LoginView loginView = new LoginView();
        loginController.setView(loginView);
        loginController.setServiceAuth(serviceAuth);
        loginView.setLoginController(loginController);
        loginView.setPrompt(prompt);

        MainController mainController = new MainController();
        MainView mainView = new MainView();
        mainView.setServiceAuth(serviceAuth);
        mainView.setPrompt(prompt);
        mainView.setMainController(mainController);
        mainController.setView(mainView);
        loginController.setNextController(mainController);

        BalanceController balanceController = new BalanceController();
        BalanceView balanceView = new BalanceView();
        balanceController.setView(balanceView);
        balanceView.setServiceAuth(serviceAuth);

```

```

        NewAccountView newAccountView = new NewAccountView();
        NewAccountController newAccountController = new NewAccountController();
        newAccountController.setServiceAuth(serviceAuth);
        newAccountController.setServiceAccount(serviceAccount);
        newAccountController.setView(newAccountView);
        newAccountView.setNewAccountController(newAccountController);

        DepositController depositController = new DepositController();
        WithdrawalController withdrawalController = new WithdrawalController();
        AccountTransactionView depositView = new AccountTransactionView();
        AccountTransactionView withdrawView = new AccountTransactionView();
        depositController.setServiceAuth(serviceAuth);
        depositController.setServiceAccount(serviceAccount);
        depositController.setView(depositView);
        withdrawalController.setServiceAuth(serviceAuth);
        withdrawalController.setServiceAccount(serviceAccount);
        withdrawalController.setView(withdrawView);
        depositView.setServiceAuth(serviceAuth);
        depositView.setPrompt(prompt);
        depositView.setTransactionController(depositController);
        withdrawView.setServiceAuth(serviceAuth);
        withdrawView.setPrompt(prompt);
        withdrawView.setTransactionController(withdrawalController);

        Map<Integer, Controller> controllerMap = new HashMap<>();
        controllerMap.put(UserOptions.GET_BALANCE.getOption(), balanceController);
        controllerMap.put(UserOptions.OPEN_ACCOUNT.getOption(), newAccountController);
        controllerMap.put(UserOptions.DEPOSIT.getOption(), depositController);
        controllerMap.put(UserOptions.WITHDRAW.getOption(), withdrawalController);

        mainController.setControllerMap(controllerMap);

        loginController.init();

```

```

    }
}

```

